

Teoría de la semana 2

Recorridos componentes conexas y caminos mínimos

Esta guía explica cómo explorar un grafo, identificar sus partes conectadas y encontrar caminos mínimos. La elección del algoritmo depende de la pregunta: DFS y BFS resuelven accesibilidad; BFS minimiza el número de aristas; Dijkstra minimiza la suma de pesos cuando todos son no negativos.

El objetivo es poder resolver una pregunta de teoría, seguir un algoritmo sobre un dibujo y justificar el coste de una implementación. Los diagramas reconstruyen ejemplos de AA02 y añaden ejemplos de refuerzo. Las tablas muestran cómo cambian las estructuras de datos en cada paso.

Ruta de estudio

Bloque	Páginas	Objetivo
Recorridos y DFS	2 a 8	Seguir la recursión y analizar código
BFS y caminos sin pesos	9 a 12	Trabajar con la cola y reconstruir rutas
Componentes y Flood Fill	13 a 16	Separar conectividad y accesibilidad
Caminos con pesos y Dijkstra	17 a 24	Relajar aristas y usar prioridades
Ejercicios y soluciones	25 a 28	Practicar y comprobar el razonamiento
Repaso y referencias	29 a 30	Preparar respuestas de examen

Cómo trabajar. En un recorrido, escribe el origen y el orden de vecinos antes de empezar. En BFS, anota la cola con su frente a la izquierda. En Dijkstra, distingue las distancias provisionales de las definitivas y registra también los predecesores.

Notación de esta guía

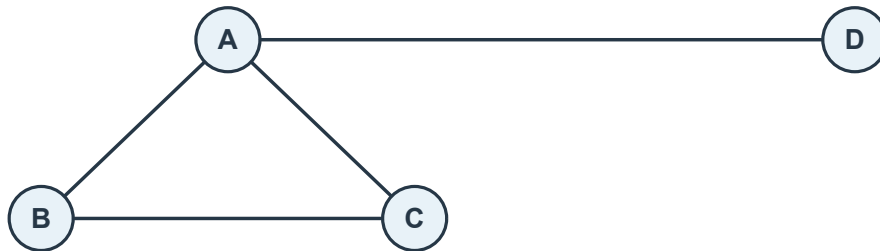
Usamos $v = |V|$ para el número de vértices y $e = |E|$ para el número de aristas. Si solo se explora lo alcanzable, v_r y e_r se refieren a esa parte del grafo. Suponemos etiquetas y operaciones aritméticas de tamaño acotado, salvo indicación contraria.

[A] identifica páginas de Semana 2/AA02.pdf; [G], secciones del libro Grafos.pdf. Las páginas PDF se cuentan desde el inicio del archivo. Los ejercicios añadidos sirven para estudiar y no son una predicción del examen.

1 Recorridos caminos y ciclos

Un **recorrido** desde x hasta y es una sucesión $x = u_0, u_1, \dots, u_k = y$ en la que cada pareja consecutiva está conectada. Puede repetir vértices y aristas. En un grafo dirigido, cada paso debe respetar la orientación del arco.

Un **camino** es un recorrido sin vértices repetidos, siguiendo la terminología de AA02. Un ciclo simple vuelve al vértice inicial sin repetir los demás. El primer y el último vértice coinciden; esa repetición es la excepción que permite cerrar el ciclo.



En el grafo dibujado, A-B-C-A-D es un recorrido: repite A. A-D es un camino. A-B-C-A es un ciclo. La longitud de un recorrido sin pesos es el número de aristas: k aristas conectan una secuencia de $k + 1$ vértices.

Tres preguntas distintas

Accesibilidad. ¿Existe algún camino de s a t ? Basta encontrar uno, aunque sea largo. DFS y BFS son adecuados.

Distancia sin pesos. ¿Qué camino utiliza menos aristas? BFS da esa garantía. DFS puede encontrar primero una ruta más larga.

Distancia ponderada. ¿Qué camino tiene menor suma de pesos? Si todos los pesos son no negativos, Dijkstra permite resolverlo. Contar aristas no basta cuando sus costes son distintos.

Grafo original y árbol de exploración

El grafo de entrada contiene todas las relaciones. El árbol de exploración conserva solo la arista por la que se descubre cada vértice distinto del origen. Puede omitir muchas aristas sin que se hayan ignorado durante la búsqueda.

Si se alcanzan r vértices desde una raíz, el árbol tiene $r - 1$ aristas. Con varios orígenes para cubrir componentes diferentes, se obtiene un bosque. El orden de vecinos puede cambiar el árbol; la accesibilidad y las distancias mínimas sin pesos no dependen de ese orden.

Para el examen. No uses «visitado» sin precisar si significa descubierto o completamente procesado. Muchos errores aparecen al tratar ambas situaciones como si fueran iguales.

Referencia: [A, pp. 3 a 5]; [G, capítulo 22].

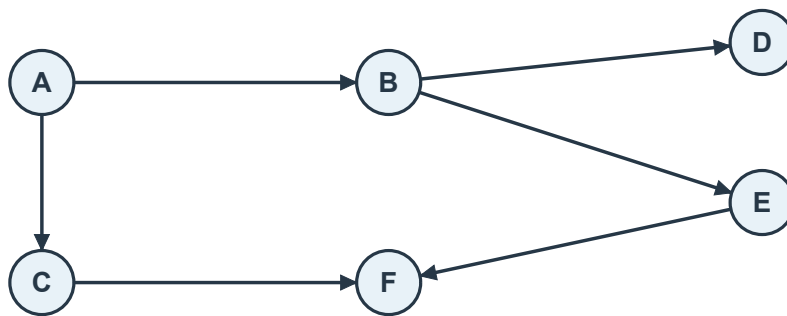
2 El DFS recursivo de AA02

DFS, búsqueda en profundidad, explora una rama antes de volver a los vecinos pendientes. La recursión proporciona una pila que recuerda las llamadas que aún no han terminado.

```
graph = {
    'A': ['B', 'C'], 'B': ['D', 'E'],
    'C': ['F'],      'D': [],
    'E': ['F'],      'F': []
}
visited = set()

def dfs(visited, graph, node):
    if node not in visited:
        print(node)
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

dfs(visited, graph, 'A')
```



La salida es **A, B, D, E, F, C**. F se alcanza primero desde E. Cuando después se explora C, se vuelve a llamar a la función con F, pero la condición inicial impide imprimirlo o expandirlo otra vez.

Lo que hace cada parte

`node not in visited` decide si hay trabajo nuevo. `visited.add(node)` marca el nodo antes de explorar sus vecinos. El bucle recorre todas las relaciones salientes. Una llamada a un vecino ya visitado retorna sin recorrer su lista.

El diccionario es una representación dirigida: por ejemplo, A incluye B, pero B no incluye A. Cada vértice de destino debe tener su propia entrada, aunque su lista sea vacía.

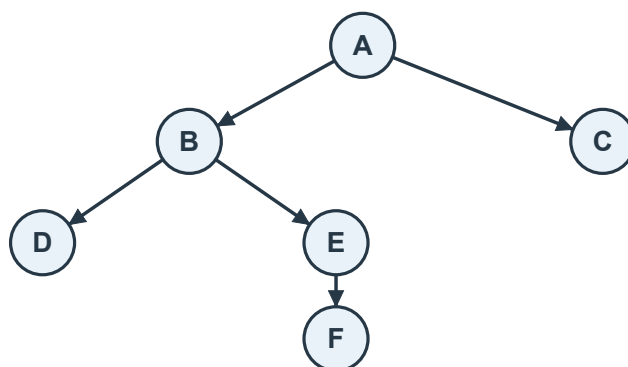
Evitar ciclos. Si existiera F-A, la vuelta a A se descartaría porque A ya estaría marcado. Marcar al final permitiría volver a entrar en un vértice cuya llamada todavía está activa y podría impedir la terminación.

Referencias: [A, pp. 8 a 10 y 12]; [G, sección 22.3].

3 Seguir la pila y construir el árbol DFS

Usamos el mismo grafo y el mismo orden de listas de la página anterior. La pila que se muestra es la de llamadas activas, desde la raíz a la llamada actual; no una lista de todos los vértices visitados.

Acción	Pila activa	Descubrimientos acumulados
Descubrir A	A	A
Entrar en B	A B	A B
Entrar en D	A B D	A B D
D termina y se entra en E	A B E	A B D E
Entrar en F	A B E F	A B D E F
Termina B y se entra en C	A C	A B D E F C
C llama a F ya visitado	A C F y retorno inmediato	No cambia



Las aristas del árbol son A-B, B-D, B-E, E-F y A-C. La relación C-F existe en el grafo original, pero no es una arista del árbol porque F ya había sido descubierto.

Descubrimiento y finalización

Imprimir antes del bucle da el orden de descubrimiento, llamado preorden en árboles. Imprimir después del bucle da el orden de finalización: **D, F, E, B, C, A** para este ejemplo.

El libro distingue blanco, gris y negro: no descubierto, activo y terminado. Visitados reúne los dos últimos estados; distinguir una llamada activa de un nodo terminado requiere información adicional.

Comprobar si un orden es posible

Un orden DFS no puede abandonar un vértice que conserva vecinos no descubiertos. Primero completa esa rama o regresa cuando ya no quedan vecinos nuevos.

Si A empieza por C, F se descubre desde C y cambian la secuencia y el árbol. Ambas exploraciones son válidas si no se fija un orden de vecinos.

Truco de examen. Separa dos columnas: «descubrimiento» y «retorno». Para dibujar el árbol, añade una arista únicamente cuando llegues por primera vez a un vértice.

Referencias: [A, pp. 9, 22 a 24]; [G, sección 22.3].

4 DFS iterativo con una pila explícita

La pila de llamadas puede sustituirse por una estructura LIFO: el último elemento añadido es el primero que se extrae. Esta versión marca al extraer y descarta las entradas repetidas **antes de volver a recorrer sus vecinos**.

```
def dfs_iterativo(adj, origen):
    visitados = set()
    pila = [origen]
    orden = []
    while pila:
        u = pila.pop()
        if u in visitados:
            continue
        visitados.add(u)
        orden.append(u)
        for w in reversed(adj[u]):
            if w not in visitados:
                pila.append(w)
    return orden
```

`reversed` invierte el orden de inserción para que el primer vecino de la lista quede arriba de la pila. Con las listas de AA02, esta versión descubre A, B, D, E, F, C, igual que la recursiva mostrada.

Cuándo pueden aparecer duplicados

Un vértice pendiente aún no está marcado. Puede entrar varias veces en la pila desde vecinos diferentes. Al extraer una copia vieja, `continue` impide expandirla otra vez. Cada lista de adyacencia se recorre una vez, y las inserciones y extracciones están acotadas por $O(v + e)$.

Con consultas esperadas constantes en el conjunto, el tiempo es $O(v + e)$ para un recorrido completo. Si la función se ejecuta solo desde un origen, depende de la parte alcanzable.

Espacio. Esta versión sencilla puede acumular $O(e)$ entradas pendientes, además de $O(v)$ visitados y salida. Por tanto, su auxiliar general es $O(v + e)$, mientras que la versión recursiva mantiene una pila activa de $O(v)$. No todas las implementaciones iterativas tienen la misma memoria.

Una variante con menos entradas pendientes

Marcar cada vértice al insertarlo limita la pila a $O(v)$, pero puede cambiar el árbol y el orden respecto de la recursión si se preparan varios vecinos a la vez. Para reproducir exactamente el desarrollo recursivo con $O(v)$ pila, se pueden guardar marcos con el vértice y un iterador de sus vecinos pendientes.

Para el examen. Indica cuándo marcas y dónde descartas duplicados. «Utiliza una pila» no basta para justificar ni el orden ni el coste.

Referencia: refuerzo de la implementación presentada en [A, p. 11].

5 Por qué el código iterativo no es óptimo

La página 11 pide analizar una implementación que usa una lista para visitados. Sus fragmentos relevantes son los siguientes; el bucle de vecinos queda fuera de la condición que detecta un nodo nuevo.

```
if node not in visited:
    print(node)
    visited.extend(node)

for next in graph[node]:
    if next not in visited:
        stack.extend(next)
```

Hay tres problemas independientes

Búsquedas lineales. `node not in visited` y `next not in visited` recorren una lista. Si ya contiene muchos vértices, una consulta puede costar $\Theta(v)$. Para comprobar pertenencia suele convenir un conjunto o un array de booleanos con vértices numerados.

Expansiones repetidas. Un nodo puede quedar varias veces pendiente antes de su primera extracción. Sus copias viejas vuelven a revisar todos sus vecinos porque el bucle no está protegido por la condición. Poner `continue` cuando ya está visitado elimina esas revisiones.

Uso incorrecto de `extend`. `extend('A')` parece funcionar porque añade un único carácter. `extend('Nodo1')` añade cinco caracteres, no el vértice completo. Con una etiqueta entera produce un error. Para añadir un solo elemento a una lista se utiliza `append`; en un conjunto, `add`.

El coste no se obtiene contando solo vértices

En un grafo simple completo de v vértices, una ejecución con este patrón puede generar $\Theta(v^2)$ extracciones. Revisar $\Theta(v)$ vecinos en cada una produce $\Theta(v^3)$ consultas, y una consulta lineal puede añadir otro factor v : **$\Theta(v^4)$ comparaciones** en esa familia.

Esto supone etiquetas individuales de tamaño constante y el patrón de la diapositiva. No es una fórmula universal para cualquier DFS iterativo: describe por qué esta implementación concreta puede ser mucho peor que el algoritmo habitual.

Cómo corregirla. Utiliza `set`, `append` para la pila y descarta un nodo repetido antes de su bucle. La versión de la página anterior hace esas tres cosas. Cambiar únicamente `visited` a conjunto no elimina las expansiones repetidas.

Consejo. Para contestar la pregunta de la diapositiva, explica primero estos problemas. Si te piden una complejidad, identifica el código exacto y sus hipótesis; no reutilices automáticamente $\Theta(v + e)$.

Referencia: [A, p. 11].

6 Visitados como conjunto o como lista

Las páginas 13 a 19 comparan variantes de DFS y muestran respuestas contradictorias. La manera segura de decidir es contar **las consultas de pertenencia**, no solo los vértices que finalmente se expanden.

Condición al entrar en la función

```
def dfs(u):
    if u not in visitados:
        visitados.add(u)
        for w in adj[u]:
            dfs(w)
```

Se expande cada vértice una vez, pero se llama a la función por cada entrada de vecino. La condición inicial también se ejecuta al recibir un vértice ya visitado. En una exploración desde un origen hay $1 + e_r$ llamadas en un grafo dirigido, o $1 + 2e_r$ en uno no dirigido.

Con un conjunto y hashing de coste esperado constante, el tiempo es $\Theta(v_r + e_r)$. Con una lista y append, cada prueba puede costar $O(v_r)$, dando una cota $O(v_r \cdot (1 + e_r))$. En grafos alcanzables densos puede ser $\Theta(v_r^3)$.

Condición antes de la llamada recursiva

```
def dfs(u):
    visitados.add(u)
    for w in adj[u]:
        if w not in visitados:
            dfs(w)
```

Ahora solo se llama a vértices nuevos, pero la pertenencia se comprueba por cada vecino. Con conjunto mantiene el tiempo lineal. Con lista, para una exploración desde un origen, una cota es $O(v_r + v_r \cdot e_r)$. La llamada inicial debe recibir un origen nuevo.

La aclaración importante de la página 16

La afirmación $O(v^2 + e)$ no es una cota general del primer código al cambiar el conjunto por una lista. Un vértice ya visitado puede recibirse muchas veces y cada llamada vuelve a hacer una búsqueda lineal. En un grafo completo, $e = \Theta(v^2)$ y ese trabajo puede ser $\Theta(v^3)$, que no cabe en $O(v^2 + e)$.

Si además recorres todos los vértices para iniciar componentes, hay hasta v pruebas exteriores. Con lista pueden añadir $O(v^2)$. Una cota general segura para cubrir todo el grafo es $O(v^2 + ve)$, sin las expansiones repetidas del código iterativo anterior.

No memorices «lista significa v^2 ». Escribe: número de pruebas multiplicado por coste de cada prueba. Un array booleano de tamaño v ofrece acceso constante determinista si puedes indexar los vértices.

Referencias: [A, pp. 13 a 19]; [G, sección 22.3].

7 Justificar la complejidad de un recorrido

Con listas de adyacencia y comprobaciones constantes de visitados, se cuenta una expansión por vértice y una revisión por entrada de vecino. En un grafo dirigido las listas suman e entradas; en uno no dirigido, $2e$.

$T(v, e) = \Theta(v) + \Theta(e) = \Theta(v + e)$ para recorrer todo el grafo. La recursión no multiplica automáticamente el coste por v : cada llamada nueva aporta su propia lista al recuento total.

Por qué no siempre puedes dejar solo e

Las páginas 20 y 21 simplifican el coste a $\Theta(e)$. Esto es válido cuando $v = O(e)$, por ejemplo en un grafo no dirigido conexo con al menos dos vértices, porque $e \geq v - 1$. No vale para un grafo arbitrario con muchos vértices aislados.

Un grafo con mil vértices y ninguna arista necesita $\Theta(v)$ trabajo si debes inicializarlos o contarlos como componentes. Escribir $\Theta(e)$ daría cero dependencia del número de vértices. La identidad $\sum d(u) = 2e$ no dice que $v = 2e$: suma grados, no cuenta vértices.

Cambiar la representación cambia el coste

Representación	Enumerar vecinos de u	Recorrido completo habitual
Listas de adyacencia	$\Theta(d(u))$	$\Theta(v + e)$
Matriz de adyacencia	$\Theta(v)$	$\Theta(v^2)$
Lista global de aristas sin índice	$\Theta(e)$	$O(v + ve)$

La tercera fila supone barrer toda la lista para buscar los vecinos de cada vértice. Si $e > 0$ y se hace el barrido completo en cada expansión, el trabajo de esos barridos es $\Theta(ve)$. Inicializar o enumerar vértices añade $\Theta(v)$.

Espacio y alcance de la búsqueda

DFS recursivo mantiene $O(v)$ visitados y hasta $O(v)$ llamadas activas. BFS puede mantener $O(v)$ distancias, predecesores y cola. Es espacio auxiliar; la representación de entrada se suma solo si se pide memoria total.

Una búsqueda desde s puede alcanzar únicamente una parte del grafo. Si no hay inicialización global, el tiempo depende de v_r y e_r . Si reservas arrays para todos los vértices, añade $\Theta(v)$, aunque luego solo explores unos pocos.

Respuesta de examen. «Cada vértice se expande una vez; las longitudes de todas las listas suman e o $2e$. Por tanto, el recorrido completo cuesta $\Theta(v + e)$, bajo comprobaciones constantes de visitados».

Referencias: [A, pp. 20 y 21]; [G, pp. PDF 12 y 21].

8 BFS y el significado de descubrir

BFS, búsqueda en anchura, utiliza una cola FIFO. Explora primero todos los vértices a distancia 1, después los de distancia 2, y así sucesivamente. La distancia se mide en número de aristas.

```
from collections import deque

def bfs(adj, origen):
    dist = {u: float('inf') for u in adj}
    prev = {u: None for u in adj}
    dist[origen] = 0
    cola = deque([origen])
    while cola:
        u = cola.popleft()
        for w in adj[u]:
            if dist[w] == float('inf'):
                dist[w] = dist[u] + 1
                prev[w] = u
                cola.append(w)
    return dist, prev
```

Se supone que todos los destinos tienen entrada en adj y que el origen existe. $\text{dist}[w]$ deja de ser infinito en el momento del descubrimiento, **antes de encolar w**. No se espera a extraerlo para marcarlo.

Por qué importa marcar al encolar

Dos vértices de la misma capa pueden tener un vecino común. Si ese vecino siguiera sin marcar hasta salir de la cola, podría entrar repetidamente y complicar tanto el árbol como el análisis. En la versión mostrada, cada vértice entra y sale como máximo una vez.

`deque.popleft()` extrae por el principio en tiempo constante. `list.pop(0)` desplaza los elementos restantes y puede añadir trabajo cuadrático. El tiempo de esta BFS es $O(v + e_p)$, incluyendo la inicialización de todos los vértices; $O(v + e)$ es la cota general.

Qué garantiza y qué no

La cola procesa las capas en orden. Cuando se descubre w desde u , se asigna la menor distancia posible en número de aristas. Si existiera una ruta más corta, w habría sido descubierto desde una capa anterior.

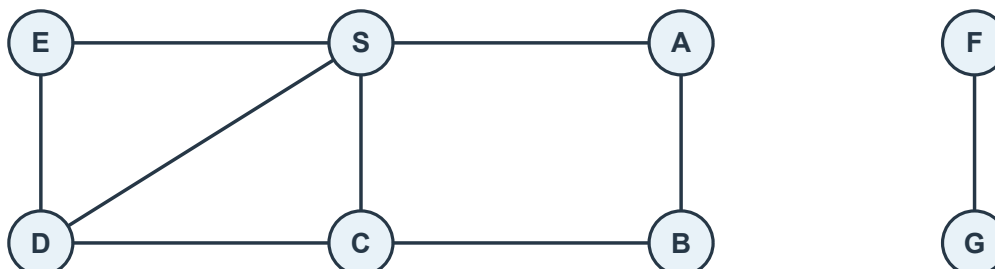
Los predecesores dependen del orden de vecinos cuando hay varias rutas mínimas. Las distancias no. BFS también funciona en grafos dirigidos, siguiendo únicamente los arcos salientes.

No confundas con DFS. El título de la página 35 contiene «profundidad», pero el contenido describe BFS por anchura y con cola. Es esa estructura y el orden por capas lo que debes reconocer.

Referencias: [A, pp. 25 a 42]; [G, sección 22.2].

9 Resolver la cola del ejercicio de clase

La página 43 contiene este grafo no dirigido. F y G forman una componente separada. Tomamos origen S y visitamos vecinos en orden alfabético; la diapositiva no fija ese orden, por lo que es necesario indicarlo.



Las listas son S: [A, C, D, E]; A: [B, S]; B: [A, C]; C: [B, D, S]; D: [C, E, S]; E: [D, S]; F: [G]; G: [F].

Nodo procesado	Cola después de procesarlo	Nuevos descubrimientos
Inicio	[S]	S con distancia 0
S	[A, C, D, E]	A, C, D y E con distancia 1
A	[C, D, E, B]	B con distancia 2
C	[D, E, B]	Ninguno
D	[E, B]	Ninguno
E	[B]	Ninguno
B	[]	Ninguno

El orden de descubrimiento y procesamiento en este ejemplo es **S, A, C, D, E, B**. Las aristas del árbol BFS son S-A, S-C, S-D, S-E y A-B. Los vértices F y G permanecen con distancia infinita y sin predecesor.

Cómo escribir una traza sin perderse

Saca un único vértice del frente, revisa sus vecinos en el orden acordado y añade solo los no descubiertos al final. Después registra la cola. No mezcles una fila «antes de extraer» con otra «después de expandir» sin avisar.

B también puede alcanzarse mediante S-C-B, con longitud 2. Como A se procesa antes que C, se guarda A como predecesor. Un cambio de orden podría guardar C sin cambiar la distancia mínima.

Comprobación. Las distancias de los vértices pendientes en la cola no disminuyen al avanzar de izquierda a derecha. Si has colocado un vértice de una capa posterior antes que otro de una capa anterior, revisa las inserciones.

Referencias: [A, p. 43]; [G, sección 22.2].

10 Reconstruir un camino mínimo

Las distancias indican el coste mínimo, pero no contienen por sí solas la secuencia de vértices. El diccionario `prev` guarda el vértice desde el que se descubrió cada destino. Se recorre hacia atrás y después se invierte la secuencia.

```
def reconstruir(prev, dist, origen, destino):
    if dist[destino] == float('inf'):
        return None
    camino = []
    u = destino
    while u is not None:
        camino.append(u)
        u = prev[u]
    camino.reverse()
    return camino
```

Este código supone que `prev` y `dist` proceden de una BFS o un Dijkstra correctos iniciados en origen, que `prev[origen]` es `None` y que destino pertenece al grafo. La cadena de predecesores de cualquier destino alcanzable debe terminar en origen.

Ejemplo con el grafo de la clase

Para B, los predecesores son `prev[B] = A` y `prev[A] = S`. La cadena hacia atrás es B, A, S; al invertirla sale **S, A, B**. Su longitud es 2 aristas, y contiene 3 vértices. Para F no hay camino desde S, por lo que se devuelve `None`.

Si origen y destino coinciden, la ruta es `[origen]`, con longitud 0. No confundas esta situación con un destino inaccesible: ambos pueden tener predecesor `None`, pero sus distancias son diferentes.

Cuándo terminar una búsqueda

Si solo buscas un destino en BFS, puedes detenerte al descubrirlo: su distancia ya es mínima. También puedes detenerte al extraerlo, una condición más fácil de compartir con la explicación de Dijkstra. Para obtener las distancias de todos los alcanzables, continúa hasta vaciar la cola.

En Dijkstra **no** basta con descubrir el destino, porque su distancia puede mejorar. La parada segura se hace al extraer una entrada vigente de distancia mínima para ese destino, bajo pesos no negativos.

Coste de recuperar la ruta

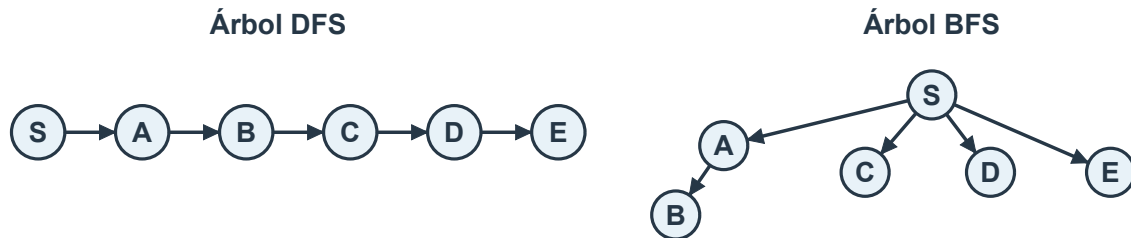
Si el camino tiene k vértices, reconstruirlo cuesta $\Theta(k)$ tiempo y $\Theta(k)$ memoria para la salida. Como un camino simple tiene como máximo v vértices, el coste es $O(v)$. El coste de encontrarlo mediante BFS se analiza aparte.

Truco. En vez de copiar un camino completo en cada inserción a la cola, guarda un predecesor por vértice y reconstruye al final. Así se mantiene el análisis habitual y se evita repetir copias de rutas parciales.

Referencias: [A, p. 44 y p. 70]; [G, secciones 22.2 y capítulo 24].

11 Comparar DFS y BFS sobre el mismo grafo

Desde S en el ejercicio de la página 43, con vecinos en orden alfabético, DFS recursivo descubre S, A, B, C, D, E. BFS descubre S, A, C, D, E, B. Ambos alcanzan los mismos seis vértices.



El árbol DFS llega a E por S-A-B-C-D-E, con cinco aristas. El árbol BFS usa directamente S-E, con una. La diferencia muestra por qué una ruta encontrada por DFS no tiene por qué ser mínima.

Aspecto	DFS	BFS
Estructura pendiente	Pila o recursión	Cola FIFO
Regla de exploración	Profundizar y retroceder	Capas de distancia creciente
Encontrar accesibles	Sí	Sí
Mínimo número de aristas	Sin garantía general	Sí
Recorrido con listas y pruebas constantes	$\Theta(v + e)$ completo	$\Theta(v + e)$ completo
Espacio auxiliar habitual	$O(v)$ recursivo	$O(v)$

Todos los árboles DFS del ejercicio 43

Con el orden alfabético acordado, cada fila siguiente es una cadena: sus parejas consecutivas forman las aristas del árbol correspondiente.

Origen	Orden DFS y cadena del árbol
A	A B C D E S
B	B A S C D E
C	C B A S D E
D	D C B A S E
E	E D C B A S
S	S A B C D E
F	F G
G	G F

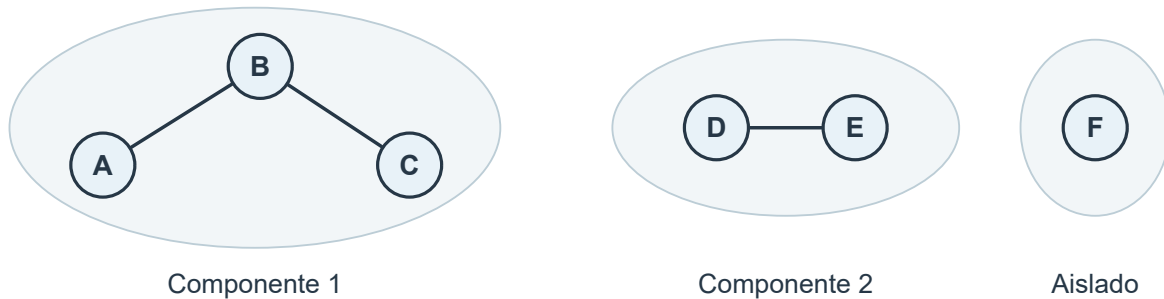
Cada fila inicia una búsqueda con visitados vacío. Compartir visitados produciría un bosque de cobertura, no búsquedas independientes desde cada nodo.

Referencias: [A, pp. 35 y 43]; [G, secciones 22.2 y 22.3].

12 Componentes conexas en grafos no dirigidos

Un grafo no dirigido es conexo si cualquier pareja de vértices está unida por un camino. Una componente connexa es un grupo **maximal** con esa propiedad: no puedes añadir un vértice exterior y conservar la conectividad del grupo.

Maximal no significa «el grupo más grande de todo el grafo». Puede haber varias componentes de tamaños distintos. Un vértice aislado forma una componente de tamaño 1. El grafo con un único vértice es conexo por la convención habitual.



En el grafo de la figura hay tres componentes: {A, B, C}, {D, E} y {F}. Dentro de cada grupo existe algún camino entre sus vértices; no tiene que haber una arista directa entre cada pareja.

Encontrarlas con exploraciones sucesivas

```
def componentes(adj):
    id_comp = {}
    numero = 0
    for s in adj:
        if s in id_comp:
            continue
        id_comp[s] = numero
        pila = [s]
        while pila:
            u = pila.pop()
            for w in adj[u]:
                if w not in id_comp:
                    id_comp[w] = numero
                    pila.append(w)
        numero += 1
    return numero, id_comp
```

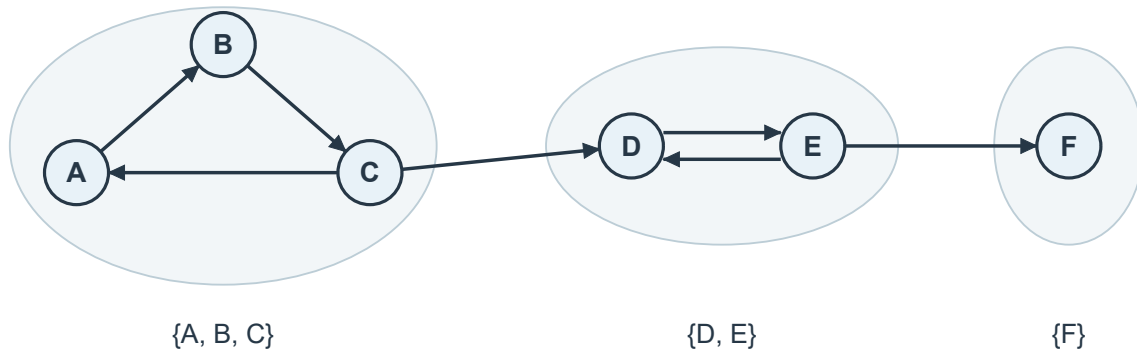
Se marca al insertar y se comparte el mapa entre todas las búsquedas. Cada nueva raíz no marcada identifica una componente. Dos vértices están conectados si tienen el mismo identificador.

Coste. El bucle exterior recorre v vértices. Cada uno se inserta y expande una vez, y las listas suman $2e$ entradas. El tiempo es $\Theta(v + e)$ con consultas constantes, y el espacio auxiliar $O(v)$. No se multiplica ese coste por el número de componentes: el trabajo se reparte entre ellas.

Referencias: [A, pp. 45 a 47]; [G, capítulo 22].

13 Conectividad débil y fuerte

En un grafo dirigido, llegar de u a w no implica poder volver de w a u . Por eso no se puede trasladar sin más la definición de componente del caso no dirigido.



En este ejemplo A , B y C forman un ciclo dirigido. D y E pueden alcanzarse mutuamente. De E sale un arco hacia F . Las componentes fuertemente conexas son $\{A, B, C\}$, $\{D, E\}$ y $\{F\}$.

Conectividad fuerte

El grafo es fuertemente conexo si para toda pareja u, w hay un camino dirigido de u a w y otro de w a u . Una componente fuertemente conexa es un grupo maximal con esa propiedad. F es una componente por sí solo, aunque no pueda regresar a otros vértices.

Conectividad débil

Se ignoran las flechas y se estudia la conectividad del grafo resultante. En la figura todos los vértices forman una sola componente débil, aunque existen tres componentes fuertes.

Una conectividad fuerte implica conectividad débil. La recíproca no es cierta: $A \rightarrow B$ con un único arco A hacia B es débilmente conexo y no fuertemente conexo.

Por qué una sola DFS no resuelve lo mismo

Desde A se alcanzan todos los vértices de la figura. Eso no demuestra que todos puedan volver a A . Un único árbol DFS puede incluir varias componentes fuertes; los grupos obtenidos iniciando búsquedas desde vértices pendientes no son, en general, las componentes fuertes.

Para encontrar componentes débiles, se puede construir la versión sin orientaciones y usar el algoritmo no dirigido. Para las fuertes hacen falta algoritmos específicos. Como refuerzo, Kosaraju utiliza dos DFS, el grafo transpuesto y el orden de finalización; el coste con listas es $\Theta(v + e)$. Su desarrollo completo no aparece en esta clase y no es necesario para entender las definiciones.

Truco de examen. Para comprobar un grupo fuerte pequeño, elige un vértice r : verifica que desde r llegas a todos y que todos pueden llegar a r . La segunda comprobación se puede hacer buscando desde r en el grafo con flechas invertidas.

Referencias: [A, pp. 48 a 50]; refuerzo [G, sección 22.5].

14 Flood Fill y conectividad en una imagen

Una imagen o tablero puede verse como un grafo implícito: cada celda válida es un vértice y los movimientos permitidos definen sus vecinos. No hace falta construir una lista de aristas para poder explorarlo.

Flood Fill identifica la región conectada a una celda inicial, normalmente restringida a celdas del mismo color o valor. Es la operación que permite rellenar una zona de una imagen sin atravesar sus límites.

Cuatro vecinos u ocho vecinos

Con conectividad 4 se consideran arriba, abajo, izquierda y derecha. Con conectividad 8 se añaden las cuatro diagonales. La elección cambia qué celdas forman una misma región.

Conectividad 4			
1	0	0	0
0	1	1	0
0	0	1	0
1	0	0	1

1 celda alcanzada

Conectividad 8			
1	0	0	0
0	1	1	0
0	0	1	0
1	0	0	1

5 celdas alcanzadas

Verde indica la región alcanzada desde la esquina superior izquierda

En ambos tableros las celdas con 1 son candidatas. Desde la esquina superior izquierda, con cuatro vecinos solo se alcanza esa celda. Con ocho vecinos se alcanzan cinco: la diagonal conecta con el grupo central y con la esquina inferior derecha. La celda inferior izquierda sigue aislada.

La región depende también de una condición

No basta con que dos posiciones sean vecinas. En un relleno por color solo conectamos celdas con el valor original. En un laberinto la condición puede ser «no es una pared»; en una imagen binaria, «es un píxel de primer plano».

Cambiar esa condición cambia el grafo que estás explorando. Debes fijarla antes de contar regiones o interpretar una ruta.

Complejidad

Una cuadrícula de m filas y n columnas tiene mn posiciones y cada una tiene a lo sumo 4 u 8 vecinos. Como ese límite es constante, explorar toda la cuadrícula cuesta $O(mn)$, y explorar una región de k celdas cuesta $O(k)$ si no hay una preparación global costosa.

Copiar toda la imagen o inicializar una matriz global de visitados añade $\Theta(mn)$, aunque la región sea pequeña. Una implementación recursiva puede acumular $O(k)$ llamadas; una iterativa usa una cola o pila de hasta $O(k)$.

Para el examen. Explica por separado la condición de inclusión, la conectividad elegida, cómo marcas las celdas y qué tamaño estás usando en el coste.

Referencias: [A, pp. 51 a 54].

15 Implementar Flood Fill sin repetir celdas

Esta versión usa cuatro vecinos y modifica la imagen. El nuevo color sirve de marca, de modo que no se necesita una matriz adicional de visitados. Se supone una matriz rectangular no vacía y una posición inicial válida.

```
from collections import deque

def flood_fill(imagen, fila, columna, nuevo):
    anterior = imagen[fila][columna]
    if anterior == nuevo:
        return 0
    m, n = len(imagen), len(imagen[0])
    cola = deque([(fila, columna)])
    imagen[fila][columna] = nuevo
    cambiadas = 1
    pasos = [(1, 0), (-1, 0), (0, 1), (0, -1)]
    while cola:
        f, c = cola.popleft()
        for df, dc in pasos:
            nf, nc = f + df, c + dc
            if 0 <= nf < m and 0 <= nc < n:
                if imagen[nf][nc] == anterior:
                    imagen[nf][nc] = nuevo
                    cambiadas += 1
                    cola.append((nf, nc))
    return cambiadas
```

Por qué se recolorea antes de insertar

Una celda puede ser vecina de varios puntos de la región. Al cambiar su color antes de encolarla, deja de cumplir la condición `== anterior`, y no vuelve a entrar en la cola. Cada celda recoloreada se procesa una vez.

El caso `anterior == nuevo` se trata al principio. Si no se hiciera, recolorear no marcaría nada y las mismas posiciones podrían insertarse repetidamente. Aquí el resultado es 0 porque no se ha cambiado ningún color, no porque la región sea vacía.

Coste y variantes

Si se cambian k celdas, el tiempo es $\Theta(k)$ y el auxiliar $O(k)$ por la cola. La propia imagen es la entrada modificada. Si quieres conservarla, una copia completa cuesta $\Theta(mn)$ tiempo y espacio adicionales.

Para ocho vecinos cambia pasos por los ocho desplazamientos. Para solo contar regiones sin modificar la imagen, usa visitados y conserva el valor original. La idea de expansión es la misma; las estructuras y el coste de preparación deben indicarse.

Error frecuente. Revisar vecinos sin comprobar los límites. En Python, un índice negativo puede acceder al lado contrario de la matriz y producir conexiones que el modelo no permite.

Referencia: desarrollo práctico del concepto de [A, p. 53].

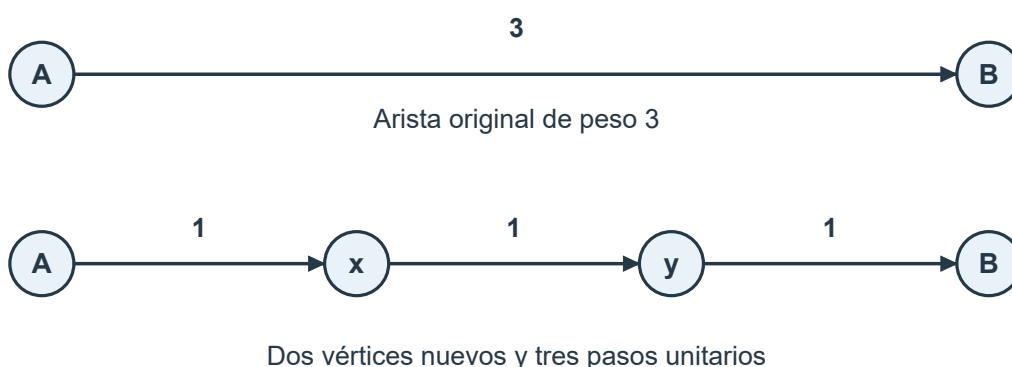
16 Por qué los pesos cambian el problema

El camino más corto puede significar menos aristas o menor suma de pesos. Cuando cada arista cuesta una unidad, ambas cosas coinciden. Si todos los costes son una misma constante positiva, basta multiplicar la distancia BFS por esa constante.

Con costes diferentes, una ruta con más aristas puede ser más barata. Una conexión directa s-t de coste 10 pierde frente a s-a-t con costes 1 y 1: dos aristas, pero coste total 2.

Convertir pesos enteros a pasos unitarios

Las páginas 59 y 60 proponen sustituir una arista de peso w por una cadena de w aristas unitarias, añadiendo $w - 1$ vértices. Entonces BFS en el grafo ampliado representa la suma de pesos originales.



Esta transformación requiere **pesos enteros positivos**. Un peso cero no se convierte en una cadena ordinaria de cero aristas entre dos vértices diferentes, y un peso fraccionario tampoco encaja directamente en esa construcción.

Si hay e aristas originales, los nuevos tamaños son $e' = \sum w(\text{arista})$ y $v' = v + \sum (w(\text{arista}) - 1)$, usando vértices nuevos independientes para cada arista. En un grafo no dirigido cada sustitución sigue siendo una conexión sin orientación.

El problema de los pesos grandes

Una sola arista de peso un millón necesita casi un millón de vértices adicionales. El tamaño deja de depender solo de v y e : depende de los valores numéricos de los pesos. Estos valores pueden ser muy grandes aunque se escriban con pocos dígitos.

La transformación explica la intuición de avanzar por tiempos de llegada, pero puede ser muy costosa. Dijkstra trabaja sobre el grafo original y evita materializar todos esos pasos intermedios.

La transición importante. BFS utiliza una cola porque todas las extensiones cuestan lo mismo. Con pesos distintos, el siguiente vértice que debe procesarse es el que tenga menor **coste acumulado desde el origen**, no el que se insertó primero.

Referencias: [A, pp. 55 a 61]; [G, ejercicio 24.3-7].

17 La idea de Dijkstra y la relajación

Dijkstra calcula las distancias mínimas desde un origen s en un grafo dirigido o no dirigido cuyos pesos son **no negativos**. El peso cero está permitido; no es obligatorio que todos sean estrictamente positivos.

Se guarda $\text{dist}[u]$, el menor coste de una ruta a u encontrada hasta el momento. Inicialmente $\text{dist}[s] = 0$ y las demás distancias son infinito. También se guarda $\text{prev}[u]$ para reconstruir una ruta.

Relajar una arista

Si ya conocemos una ruta a u de coste $\text{dist}[u]$, continuar por (u, w) ofrece una ruta candidata de coste $\text{dist}[u] + \text{peso}(u, w)$. Se actualiza w solo si mejora:

```
candidato = dist[u] + peso
if candidato < dist[w]:
    dist[w] = candidato
    prev[w] = u
```

Por ejemplo, $\text{dist}[u] = 3$, $\text{peso}(u, w) = 5$ y $\text{dist}[w] = 10$. Pasar por u cuesta 8, así que $\text{dist}[w]$ cambia a 8 y su predecesor a u . Si el candidato fuera 12, no se modifica nada.

Provisional no significa definitivo

Un vértice descubierto tiene alguna ruta conocida, pero puede aparecer una más barata después. Dijkstra elige entre los pendientes el vértice con menor dist y lo fija cuando se extrae correctamente de la cola de prioridad.

La prioridad es el coste completo desde s . No se elige la arista más barata de todo el grafo ni el menor peso saliente. Si dos pendientes tienen distancias 8 y 10, se procesa primero el de 8, aunque su última arista haya sido más cara.

La analogía de las alarmas

Las diapositivas describen una alarma por vértice programada a su tiempo de llegada. Si se descubre una ruta más rápida, la alarma se adelanta. La próxima alarma que suena es la de menor tiempo pendiente.

Esta descripción conduce a la cola de prioridad. Aunque algunas diapositivas mencionen DFS al introducir la analogía, Dijkstra no usa el orden de una pila DFS: necesita seleccionar el mínimo global pendiente.

Qué devuelve. Las distancias desde s a los destinos alcanzables, y un árbol de predecesores para una elección de rutas mínimas. Los destinos inaccesibles quedan a infinito. No calcula por sí solo rutas entre todas las parejas.

Referencias: [A, pp. 63 a 70]; [G, sección 24.3].

18 Por qué Dijkstra necesita pesos no negativos

El invariante esencial es que los vértices ya fijados tienen su distancia mínima real desde el origen. Cuando se extrae el pendiente u de menor distancia provisional, Dijkstra debe poder fijarlo sin que una ruta futura lo mejore.

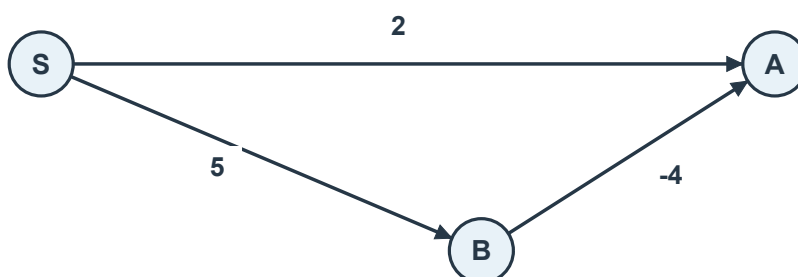
La intuición de la demostración

Supón que hubiera una ruta más barata a u . En esa ruta, toma el primer vértice y aún no fijado y su predecesor x , que sí está fijado. Cuando se procesó x , se relajó la arista hacia y , así que y ya tenía una estimación tan buena como el prefijo de esa ruta.

Como los pesos restantes son no negativos, ese prefijo no cuesta más que la ruta completa a u . Por tanto, y habría tenido prioridad menor que la supuesta distancia incorrecta de u . Eso contradice que u fuera el mínimo pendiente extraído.

La no negatividad es lo que permite comparar el coste de un prefijo con el de la ruta completa. Con una arista negativa, continuar por el camino puede reducir el coste y romper el argumento.

Un contraejemplo pequeño



Desde S se asigna $A = 2$ y $B = 5$. El Dijkstra que fija A definitivamente lo procesa antes que B . Sin embargo, $S-B-A$ cuesta $5 - 4 = 1$, mejor que 2 . Una ruta descubierta más tarde puede mejorar un vértice que ya se había dado por definitivo.

Una variante que reabre vértices puede comportarse de otra forma, pero ya no puedes aplicar sin más la prueba ni las cotas del Dijkstra estándar. Con pesos negativos debes elegir un algoritmo apropiado, como Bellman Ford; su desarrollo corresponde a otra parte de la asignatura.

Pesos cero y empates

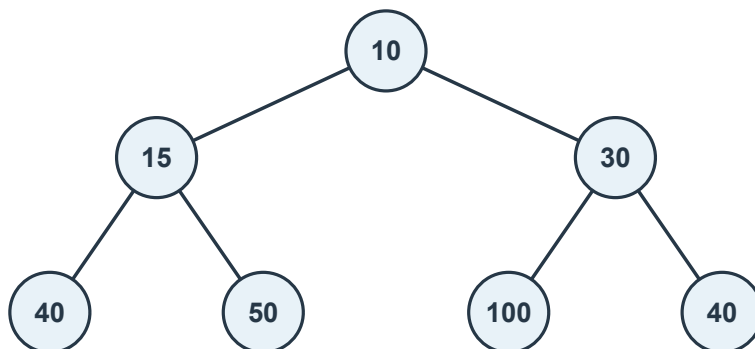
Los pesos cero sí cumplen la hipótesis. Pueden aparecer varios vértices con la misma distancia mínima y varias rutas de igual coste. Resolver el empate de una u otra manera no cambia las distancias finales.

Usar $<$ en la relajación conserva el predecesor anterior en un empate. Cambiar predecesores indiscriminadamente con $\leq$, especialmente con ciclos de coste cero, puede crear problemas al reconstruir rutas.

Referencias: [A, pp. 57 y 63]; [G, sección 24.3 y capítulo 24].

19 Cola de prioridad y min heap

Una cola de prioridad extrae el elemento con menor clave. Para Dijkstra, la clave es la distancia provisional desde el origen. Una implementación habitual utiliza un **min heap binario**.



En un min heap, cada padre tiene una clave menor o igual que la de sus hijos. El mínimo está en la raíz. Los hermanos no tienen por qué estar ordenados, ni el array que guarda el heap es una lista globalmente ordenada.

Operación	Función en Dijkstra	Coste con heap binario
Insertar	Añadir una nueva estimación	$O(\log v)$ con un registro por vértice
Extraer mínimo	Elegir el siguiente vértice	$O(\log v)$
Disminuir clave	Adelantar una distancia	$O(\log v)$ si se localiza el registro
Construir heap	Preparar todos los registros	$O(v)$

Disminuir clave requiere poder localizar el registro del vértice sin recorrer todo el heap. Normalmente se conserva una referencia o índice y se actualiza al intercambiar posiciones.

La variante práctica con heapq

heapq proporciona inserción y extracción, pero no una operación directa que localice y disminuya cualquier clave. Una técnica sencilla es insertar una pareja nueva cuando la distancia mejora y dejar la antigua dentro.

Al extraer una pareja (d, u) , si d ya no coincide con $\text{dist}[u]$, se descarta: es una alarma antigua. El heap puede guardar varios registros del mismo vértice y su tamaño depender de e , no solo de v . Por eso se debe ajustar también la cota de memoria.

Resolver empates en Python

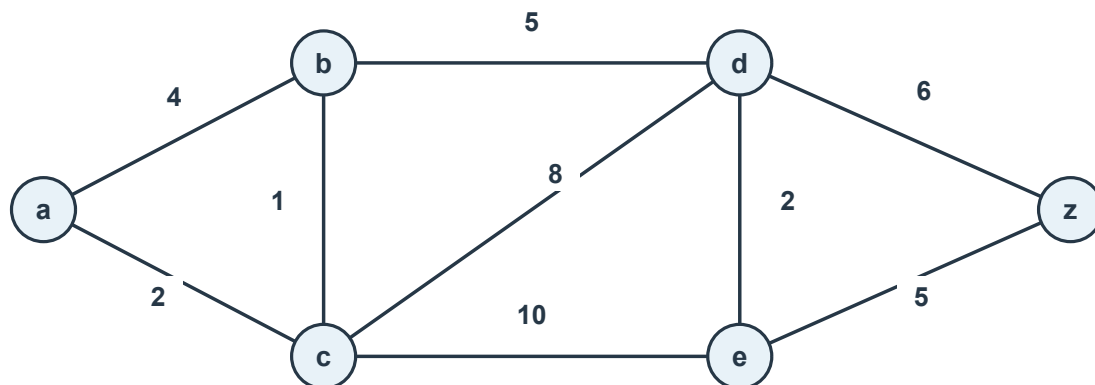
Con parejas $(\text{distancia}, \text{vertice})$, Python compara los vértices si las distancias empatan. Eso funciona con etiquetas comparables, pero puede fallar con objetos de tipos distintos. Añadir un contador único produce $(\text{distancia}, \text{contador}, \text{vertice})$ y evita comparar las etiquetas.

Error frecuente. Cambiar $\text{dist}[u]$ no cambia automáticamente una clave ya almacenada en el heap. Hay que hacer la disminución de clave real o insertar un registro nuevo y descartar el viejo después.

Referencias: [A, pp. 66 a 69]; [G, sección 24.3].

20 El ejemplo de Dijkstra de la clase

Las páginas 64 y 71 a 79 trabajan este grafo no dirigido. Usaremos el nombre z para el destino de la derecha; algunas de las tablas de las diapositivas lo llaman f. Es el mismo papel en el ejemplo.



Las aristas son a-b: 4; a-c: 2; b-c: 1; b-d: 5; c-d: 8; c-e: 10; d-e: 2; d-z: 6; e-z: 5. Al ser no dirigido, se pueden recorrer en ambos sentidos.

Primeras actualizaciones

Desde a se obtiene $b = 4$ y $c = 2$. Se elige **c**, porque su distancia provisional es menor. Pasar desde c a b ofrece $2 + 1 = 3$: mejora la ruta directa de coste 4, así que $\text{prev}[b]$ cambia de a a c.

Desde c también se descubre $d = 10$ y $e = 12$. Después se procesa b con distancia 3 y se mejora d a $3 + 5 = 8$. Al procesar d, e baja a $8 + 2 = 10$ y se descubre z con $8 + 6 = 14$.

Por qué no se usa solo el peso de la arista

La arista b-c pesa 1, pero su importancia depende del coste acumulado para llegar a sus extremos. Al salir de a se elige c porque cuesta 2 llegar a c, no porque se haya elegido globalmente la arista de peso 1.

El orden de fijación en este ejemplo es **a, c, b, d, e, z**, con distancias definitivas 0, 2, 3, 8, 10 y 14. Al procesar e, el candidato para z vale $10 + 5 = 15$ y no mejora el 14 ya conocido.

Cómo preparar la tabla a mano

Escribe una columna por vértice y una fila por extracción vigente. En cada fila identifica el mínimo pendiente, fija ese nodo y relaja sus vecinos. Si hay una mejora, cambia simultáneamente la distancia y el predecesor.

Comprobación de orientación. En un grafo dirigido solo se relajan los arcos que salen del nodo actual. No puedes añadir la conexión inversa porque el dibujo parezca parecido a este ejemplo no dirigido.

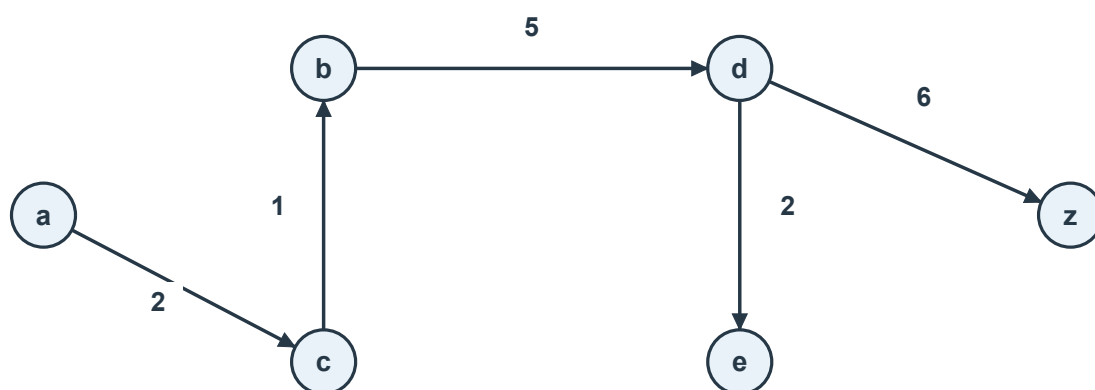
Referencia: [A, pp. 64 y 71 a 79].

21 Tabla completa y árbol de caminos mínimos

Esta tabla recoge las distancias después de procesar cada vértice del ejemplo anterior. «Fijado» indica que se ha extraído una entrada vigente con la menor prioridad pendiente.

Fijado	a	b	c	d	e	z
Inicial	0	∞	∞	∞	∞	∞
a	0	4	2	∞	∞	∞
c	0	3	2	10	12	∞
b	0	3	2	8	12	∞
d	0	3	2	8	10	14
e	0	3	2	8	10	14
z	0	3	2	8	10	14

Los predecesores finales son $\text{prev}[a] = \text{None}$, $\text{prev}[c] = a$, $\text{prev}[b] = c$, $\text{prev}[d] = b$, $\text{prev}[e] = d$ y $\text{prev}[z] = d$. Sus aristas forman este árbol de caminos mínimos:



Recuperar las rutas y comprobar el coste

La ruta a z es **a-c-b-d-z**, de coste $2 + 1 + 5 + 6 = 14$. Para e es a-c-b-d-e, de coste $2 + 1 + 5 + 2 = 10$. Sigue la cadena de predecesores hacia atrás y verifica después cada arista y su peso.

En cada vértice alcanzable distinto del origen debe cumplirse $\text{dist}[u] = \text{dist}[\text{prev}[u]] + \text{peso}(\text{prev}[u], u)$. Esta igualdad comprueba que el árbol representa las distancias registradas, aunque por sí sola no sustituye a la demostración de optimalidad.

Qué aparece en un heap con registros repetidos

Cuando b baja de 4 a 3, el registro antiguo de prioridad 4 puede seguir pendiente. Al extraerlo se descarta, porque 4 ya no coincide con $\text{dist}[b] = 3$. Lo mismo ocurre con las versiones antiguas de d = 10 y e = 12.

Estas extracciones descartadas no se añaden al orden de fijación de la tabla. Mezclar ambos tipos de extracción hace parecer que Dijkstra fija varias veces el mismo vértice.

Referencia: desarrollo completo de [A, pp. 72 a 79].

22 Dijkstra en Python con entradas antiguas

Esta versión usa listas de pares (vecino, peso) y presupone pesos no negativos. Todos los destinos y el origen deben tener entrada en adj. El contador permite empates sin comparar las etiquetas de los vértices.

```
from heapq import heappush, heappop
from itertools import count

def dijkstra(adj, origen):
    dist = {u: float('inf') for u in adj}
    prev = {u: None for u in adj}
    dist[origen] = 0
    ticket = count()
    heap = [(0, next(ticket), origen)]
    while heap:
        d, _, u = heappop(heap)
        if d != dist[u]:
            continue
        for w, peso in adj[u]:
            candidato = d + peso
            if candidato < dist[w]:
                dist[w] = candidato
                prev[w] = u
                heappush(heap, (candidato, next(ticket), w))
    return dist, prev
```

Qué resuelve la condición de descarte

Un registro viejo puede contener una distancia mayor que la estimación actual. No se debe expandir de nuevo desde esa estimación antigua. La condición `d != dist[u]` conserva únicamente las alarmas vigentes.

Solo se inserta cuando hay una **mejora estricta**. Con pesos no negativos, la primera extracción vigente de un vértice fija su distancia mínima. Las revisiones totales de vecinos siguen acotadas por las entradas de aristas alcanzables.

Guardar la información correcta

Al mejorar un destino, cambia tanto `dist` como `prev`. Para obtener el camino, usa la reconstrucción de la página 11 con estos resultados. Si solo interesa un destino, la parada segura se puede colocar después del descarte: cuando `u` sea ese destino, su `d` vigente será definitivo.

Los vértices inaccesibles nunca se insertan y conservan infinito. La inicialización de `dist` y `prev`, sin embargo, recorre todos los vértices y cuesta $\Theta(v)$.

Precisión. El código es una implementación didáctica con números y etiquetas de tamaño acotado. Las operaciones con números enormes o problemas de redondeo requieren un análisis adicional que no se utiliza en los ejercicios de esta guía.

Referencias: [A, pp. 69 y 70]; refuerzo de implementación de [G, sección 24.3].

23 Complejidad de Dijkstra y errores de examen

La complejidad depende de la cola de prioridad y de la representación. No basta con decir «Dijkstra siempre es $O(e \log v)$ ».

Implementación	Tiempo general	Memoria auxiliar habitual
Buscar el mínimo en un array y usar listas	$O(v^2 + e)$	$O(v)$
Matriz y búsqueda lineal del mínimo	$O(v^2)$	$O(v)$
Heap binario con disminuir clave	$O((v + e) \log v)$	$O(v)$
Heap con registros repetidos	$O(v + (e + 1) \log(e + 2))$	$O(v + e)$

Los factores logarítmicos se entienden para tamaños crecientes, con el tratamiento constante de grafos diminutos. La memoria de entrada se cuenta aparte.

Contar las operaciones de prioridad

En la versión con un registro por vértice hay hasta v extracciones y hasta e disminuciones de clave. Con heap binario cuestan $O(\log v)$. Inicializar y recorrer aristas añade $O(v + e)$.

En la versión de Python, cada relajación exitosa introduce un registro nuevo: hay $O(e + 1)$ inserciones y extracciones, y el heap puede llegar a $O(e + 1)$. Por eso el logaritmo natural del análisis de esa implementación es $\log(e + 2)$.

Para grafos simples, $e = O(v^2)$, de modo que $\log(e + 2) = O(\log v)$ cuando v crece. Se obtiene la cota habitual $O((v + e) \log v)$. Si todos los vértices son alcanzables desde el origen, $e \geq v - 1$ y suele abreviarse a $O(e \log v)$. Con muchos aislados, conserva el término v .

Los errores que debes vigilar

Fijar al descubrir. En el ejemplo de clase b se descubre con 4 y termina con 3. Su primera estimación no era definitiva.

Elegir una arista por su peso. Se selecciona el mínimo coste acumulado de un vértice pendiente. No es el criterio de un árbol de expansión mínimo.

Usar una cola FIFO con pesos distintos. Puede procesar antes una ruta cara. BFS no garantiza mínimo coste ponderado en ese caso.

Actualizar dist sin actualizar la prioridad. Una clave del heap no se modifica automáticamente. Debes disminuirla correctamente o insertar una entrada nueva.

Parar cuando se inserta el destino. Para Dijkstra, espera a su extracción vigente. **Aceptar pesos negativos.** Rompe la garantía general. **Olvidar prev.** Obtendrás un coste, pero no la ruta solicitada.

Referencias: [A, pp. 63 a 70]; [G, pp. PDF 76 y 77].

24 Ejercicios para preparar el examen

Responde primero sin consultar la página siguiente. Define el tamaño, el orden de vecinos y las hipótesis que necesites. Supón un grafo simple y operaciones constantes sobre etiquetas acotadas.

Ejercicio 1 Elegir algoritmo

Para cada tarea, elige DFS, BFS o Dijkstra y justifica: saber si existe alguna ruta; minimizar el número de saltos; minimizar la suma de tiempos no negativos; contar componentes de un grafo no dirigido.

Ejercicio 2 Dos condiciones de visitados

En el DFS recursivo de la página 3 se sustituye `set` por una lista y `add` por `append`. ¿Cuántas veces se comprueba pertenencia? ¿Es $O(v^2 + e)$ una cota general correcta para grafos densos alcanzables?

Ejercicio 3 Un bucle exterior

```
visitados = set()
for s in adj:
    if s not in visitados:
        explorar(s, adj, visitados)
```

`explorar` es un DFS correcto con listas de adyacencia y conjunto compartido. Justifica el tiempo total. ¿Qué cambia si cada llamada empieza con un conjunto nuevo y se explora desde todos los orígenes?

Ejercicio 4 La cola con pop cero

Se implementa BFS con una lista y `cola.pop(0)`. ¿Qué trabajo oculto aparece? Da una familia sencilla de grafos donde esos desplazamientos sean cuadráticos.

Ejercicio 5 Conectividad dirigida

En el grafo A hacia B hacia C , determina accesibilidad desde A , componentes fuertes y componentes débiles. ¿Por qué un único árbol DFS no demuestra conectividad fuerte?

Ejercicio 6 Flood Fill

En la cuadrícula de la página 15, calcula cuántas celdas con 1 alcanza la esquina superior izquierda con conectividad 4 y 8. ¿Cuántas regiones de unos hay en toda la imagen en cada modelo?

Ejercicio 7 Una distancia que cambia

Un grafo dirigido tiene $s \rightarrow a$ de peso 8, $s \rightarrow b$ de peso 2 y $b \rightarrow a$ de peso 1. Escribe las primeras actualizaciones de Dijkstra. Si el heap conserva $(8, a)$, ¿qué hay que hacer cuando se extraiga?

Ejercicio 8 Peso negativo y empate

Explica por qué el contraejemplo de la página 19 rompe la prueba de Dijkstra. Después razona si dos rutas de igual coste obligan a que todos los árboles de predecesores sean idénticos.

25 Soluciones razonadas

Ejercicio 1

Para existencia de ruta sirven **DFS o BFS**. Para mínimo número de saltos, **BFS**. Para mínimo coste con pesos no negativos, **Dijkstra**. Para componentes no dirigidas, búsquedas DFS o BFS desde cada vértice aún no descubierto, compartiendo visitados.

Ejercicio 2

La condición se ejecuta por cada llamada, incluidas las llamadas que se descartan. Desde un origen hay $1 + e_r$ llamadas en un grafo dirigido, o $1 + 2e_r$ en uno no dirigido. Con lista puede haber $\Theta(v_r)$ comparaciones por prueba y $\Theta(v_r^3)$ en familias densas. $O(v^2 + e)$ no es una cota general de ese código.

Ejercicio 3

Con visitados compartido, cada vértice se expande una vez y cada entrada de vecino se revisa una vez. El tiempo global es $\Theta(v + e)$, con consultas constantes. Si se reinicia visitados y se busca desde cada origen, se repite trabajo; una cota es $O(v \cdot (v + e))$, alcanzable en orden en familias donde cada origen llega a todo el grafo.

Ejercicio 4

$\text{pop}(\theta)$ desplaza la cola restante. En una estrella, la raíz añade $v - 1$ hojas y después se extraen una a una. Los desplazamientos suman $(v - 2) + \dots + 1 = \Theta(v^2)$, aunque $e = v - 1$ y el BFS con deque sea lineal.

Ejercicio 5

Desde A se alcanzan A, B y C. Las componentes fuertes son **{A}, {B}, {C}**: no hay regreso desde B a A ni desde C a B. Hay una única componente débil {A, B, C}. Alcanzar todos desde A no equivale a que todas las parejas se alcancen en ambos sentidos.

Ejercicio 6

Desde la esquina inicial se alcanza **1 celda** con cuatro vecinos y **5 celdas** con ocho. En toda la cuadrícula hay **4 regiones** con conectividad 4 y **2 regiones** con conectividad 8. La celda inferior izquierda permanece separada en ambos modelos.

Ejercicio 7

Tras procesar s: $a = 8$ y $b = 2$. Se extrae b y a mejora a **3**, con $\text{prev}[a] = b$. Se inserta la estimación nueva. Cuando aparezca $(8, a)$, se descarta por no coincidir con $\text{dist}[a] = 3$; no se vuelve a expandir a desde 8.

Ejercicio 8

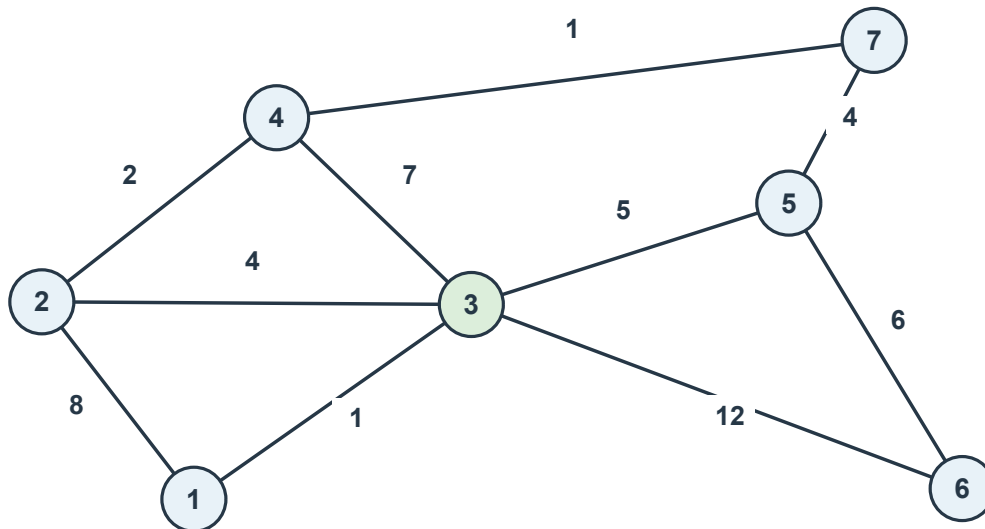
La arista negativa permite que una ruta futura reduzca una distancia ya fijada, invalidando el argumento de prefijos no decrecientes. En un empate pueden elegirse diferentes predecesores y árboles, manteniendo **las mismas distancias mínimas**. Con relajación estricta se conserva la primera ruta mínima encontrada según los desempates del algoritmo.

26 Los dos ejercicios finales de Dijkstra

Los diagramas reconstruyen las páginas 83 y 84 de AA02. Practica las distancias y los predecesores antes de consultar las soluciones. Indica el origen elegido y la regla de desempate.

Ejercicio de la página 83

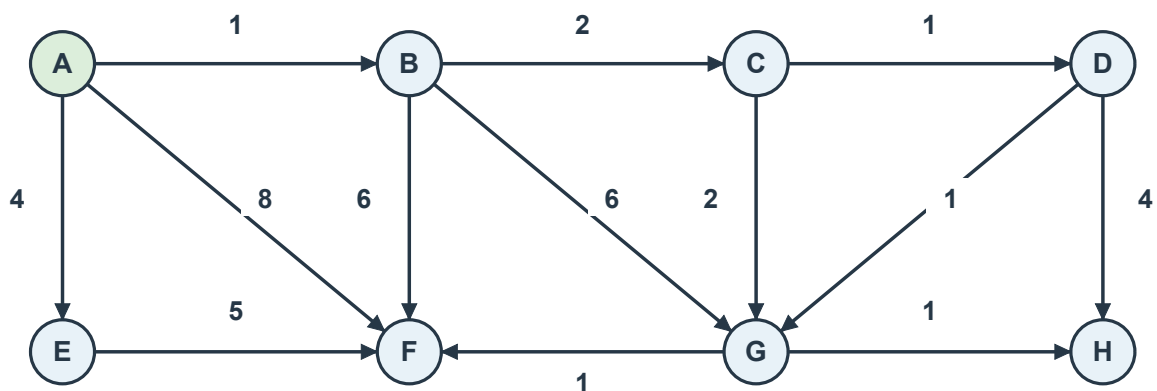
Tomamos como origen el vértice 3, que aparece sombreado en la diapositiva. El grafo es **no dirigido**. Calcula las distancias a los otros seis vértices, el orden de fijación y una ruta mínima a 6 y a 7.



Fíjate en las aristas 3-4 de peso 7, 2-4 de peso 2 y 4-7 de peso 1. Una conexión directa puede empeorar frente a una ruta con más pasos.

Ejercicio de la página 84

Tomamos A como origen y resolvemos empates por orden alfabético. El grafo es **dirigido**. Calcula las distancias finales, conserva el predecesor anterior si el candidato empata y recupera las rutas a F y H.



Comprueba las flechas: G va hacia F, D va hacia G y E va hacia F. El movimiento contrario no está permitido salvo que haya otro arco explícito.

Referencias: [A, pp. 83 y 84].

27 Soluciones de los ejercicios finales

Página 83 desde el vértice 3

Nodo	1	2	3	4	5	6	7
Distancia	1	4	0	6	5	11	7
prev	3	3	None	2	3	5	4

El orden de fijación es **3, 1, 2, 5, 4, 7, 6**. Desde 3 se descubre 4 con 7 y 6 con 12. Al procesar 2, la ruta 3-2-4 cuesta $4 + 2 = 6$ y mejora el 7. Al procesar 5, la ruta 3-5-6 cuesta $5 + 6 = 11$ y mejora el 12.

El vértice 7 se descubre primero desde 5 con $5 + 4 = 9$. Después baja a $6 + 1 = 7$ al procesar 4. No estaba fijado todavía, por lo que esa mejora es válida.

Rutas solicitadas. A 6: 3-5-6, coste 11. A 7: 3-2-4-7, coste 7. La ruta directa 3-6 cuesta 12; tener menos aristas no la hace mejor.

Página 84 desde A

Nodo	A	B	C	D	E	F	G	H
Distancia	0	1	3	4	4	6	5	6
prev	None	A	B	C	A	G	C	G

Con el desempate acordado, el orden de fijación es **A, B, C, D, E, G, F, H**. D y E empatan a 4; F y H empatan a 6. Cambiar el orden de extracción entre vértices empatados puede ser válido.

F se descubre desde A con 8, mejora desde B a $1 + 6 = 7$ y vuelve a mejorar desde G a $5 + 1 = 6$. G se descubre desde B con 7 y baja desde C a $3 + 2 = 5$. La ruta por D también cuesta $4 + 1 = 5$; como se conserva el predecesor en un empate, prev[G] sigue siendo C.

H se descubre desde D con $4 + 4 = 8$ y baja desde G a $5 + 1 = 6$. El coste final de F y H coincide, aunque las rutas terminen en vértices distintos.

Rutas solicitadas. A F: A-B-C-G-F, coste $1 + 2 + 2 + 1 = 6$. A H: A-B-C-G-H, también coste 6. Otra ruta mínima válida pasa por D antes de G; la tabla concreta utiliza C como predecesor de G.

Una comprobación final

Suma los pesos de las rutas reconstruidas. Revisa que cada paso respeta las flechas y que su coste coincide con la tabla. Si existe una entrada antigua en el heap, no la confundas con una nueva fijación del vértice.

28 Método de respuesta y repaso rápido

Si te dan un dibujo

1. Identifica dirección, pesos, origen y orden de vecinos.
2. Elige el algoritmo según la pregunta, no según el aspecto del dibujo.
3. DFS: registra llamadas y retornos, y añade una arista al árbol al descubrir un nodo nuevo.
4. BFS: muestra la cola después de cada expansión, las distancias y los predecesores.
5. Dijkstra: extrae el mínimo provisional vigente, fija y relaja; actualiza distancia y predecesor juntos.
6. Reconstruye la ruta si se pide y comprueba su longitud o suma de pesos.

Si te dan código para analizar

Cuenta consultas, expansiones, entradas de vecinos y operaciones de cola o heap. Una lista de visitados puede añadir búsquedas lineales; una cola con `pop(0)`, desplazamientos; una expansión repetida, recorridos adicionales. El código concreto puede tener peor coste que el algoritmo que intenta implementar.

No olvides la inicialización. Arrays o diccionarios para todos los vértices cuestan $\Theta(v)$. Un recorrido de toda la imagen o una copia completa cuestan $\Theta(mn)$. El espacio auxiliar se analiza por separado.

Qué algoritmo corresponde a cada pregunta

Pregunta	Elección habitual	Condición principal
Hay alguna ruta o qué es alcanzable	DFS o BFS	Respetar las flechas
Menos aristas o saltos	BFS	Sin pesos o costes unitarios
Menor suma de pesos	Dijkstra	Pesos no negativos
Componentes no dirigidas	DFS o BFS sucesivos	Compartir visitados
Regiones de una cuadrícula	Flood Fill	Fijar condición y vecindad
Componentes fuertes	Algoritmo específico	No basta una DFS simple

Ideas que debes poder explicar sin mirar

DFS profundiza con pila; BFS explora capas con cola; Dijkstra prioriza el coste acumulado. BFS marca al encolar. Dijkstra no fija al descubrir: fija al extraer el mínimo vigente. Sus distancias son provisionales hasta ese momento.

Con listas de adyacencia y pruebas constantes, un recorrido completo cuesta $\Theta(v + e)$. No se elimina v si hay muchos aislados. Una componente es maximal, no necesariamente la más grande. La fuerte exige ida y vuelta; la débil ignora direcciones.

Las respuestas más convincentes combinan resultado y motivo: «Elijo BFS porque todas las aristas tienen el mismo coste y necesito minimizar los saltos», o «Esta consulta se repite por cada entrada de vecino y cuesta $O(v)$, por eso introduce un factor adicional».

29 Aclaraciones de las diapositivas y lecturas

Puntos que conviene interpretar con precisión

Páginas 13 a 19. La comparación entre lista y conjunto debe hacerse sobre las consultas reales. La cota $O(v^2 + e)$ de la página 16 no es general para el código con llamadas por cada vecino; puede haber tiempo cúbico en grafos densos. La página 7 de esta guía detalla ambos patrones.

Páginas 20 y 21. $\sum d(u) = 2e$ cuenta incidencias. La simplificación de $\Theta(v + e)$ a $\Theta(e)$ necesita $v = O(e)$. No es válida para cubrir grafos con muchos vértices aislados.

Página 35. Su título dice profundidad, pero describe BFS con cola y por niveles. **Páginas 57 y 63.** La condición correcta de Dijkstra es peso no negativo, incluyendo cero. **Página 70.** La prioridad se basa en la distancia acumulada desde el origen, no solo en el peso de la última arista.

Páginas 72 a 79. El destino que el dibujo inicial llama z aparece como f en algunas tablas. La guía conserva z y completa su fijación. Estas aclaraciones evitan convertir un cambio de etiqueta en un vértice adicional.

Referencias principales

[A] Santi Seguí. AA02.pdf, curso 2026 a 2027. Archivo Semana 2/AA02.pdf. Recorridos: pp. 3 a 5. DFS e implementaciones: pp. 7 a 24. BFS y ejercicios: pp. 25 a 44. Componentes: pp. 45 a 50. Flood Fill: pp. 51 a 54. Caminos con pesos y Dijkstra: pp. 55 a 84.

[G] Grafos.pdf, carpeta Libros. Extracto de Introduction to Algorithms, Cormen, Leiserson, Rivest y Stein. Se han consultado las siguientes secciones para contrastar conceptos, garantías y costes:

1. Sección 22.1, representaciones de grafos: pp. PDF 4 a 8, impresas 589 a 593.
2. Sección 22.2, BFS: pp. PDF 9 a 17, impresas 594 a 602; el análisis y la independencia de las distancias respecto del orden aparecen en p. PDF 12, impresa 597.
3. Sección 22.3, DFS: desde p. PDF 18, impresa 603; análisis del coste en p. PDF 21, impresa 606.
4. Sección 22.5, componentes fuertes: especialmente p. PDF 32, impresa 617, como ampliación conceptual.
5. Capítulo 24, representación de rutas: p. PDF 62, impresa 647; sección 24.3, Dijkstra: pp. PDF 73 a 77, impresas 658 a 662, con prueba de corrección y comparación de implementaciones.

Los dibujos de ejemplos de clase se han reconstruido respetando sus conexiones y pesos. Los diagramas de recorridos, conectividad dirigida, relleno y contraejemplos añaden apoyo visual para explicar los conceptos.